

Prompt Injection 自動化測試系統實作與中小規模 LLM 防禦分析

¹ 薛文豪, ² 邱泓崴, ³ 王永煜, ⁴ 張林煌*

¹ 國立臺中教育大學 資訊工程學系
E-mail: acs111131@gm.ntcu.edu.tw

² 國立臺中教育大學 資訊工程學系
E-mail: acs111851@gm.ntcu.edu.tw

³ 國立臺中教育大學 資訊工程學系
E-mail: acs111110@gm.ntcu.edu.tw

⁴ 國立臺中教育大學 資訊工程學系
E-mail: lchang@mail.ntcu.edu.tw

摘要

大型語言模型 (LLM) 如 ChatGPT 的應用日益普及，但同時也面臨安全風險，特別是 Prompt Injection 攻擊。近期案例如台北捷運站智慧客服 Jailbreak 事件及 Google Gemini 拒絕服務事件顯示，即便具備強大安全機制仍可能因特定輸入而繞過限制，導致資訊外洩或遭受惡意攻擊。本研究設計一套自動化測試平台，透過隨機產生的 FLAG 作為敏感資訊測試標的，並模擬多種 Prompt Injection 攻擊策略，以評估中小型語言模型在防洩漏防禦方面的實際表現與穩健性。

關鍵字：大型語言模型、Prompt Injection、防禦分析、中小規模模型。

1. 前言

LLM (Large Language Model)[1] 隨著 ChatGPT[1] 的商業化與普及，逐漸進入大眾視野。隨著各類語言模型的開源與權重公開，LLM 技術被廣泛應用於不同領域。然而，隨著應用程式與網站逐步整合 LLM 模型，安全性問題也日益凸顯。例如，根據 OWASP TOP 10 for LLM 中所列，Prompt Injection 佔據了 2023-25 年度的十大 LLM 弱點榜首[2]。

這類安全漏洞已導致多起機敏資料外洩、惡意利用及繞過安全機制 (Bypass Security Rule) 的事件。近期最為知名的案例為「台北捷運站智慧客服 Jailbreak 事件」[3]，使用者透過特定輸入成功使智慧客服脫離預設規範，回應無關問題，如「如何使用 C++ 撰寫 Hello World」，暴露出 LLM 在安全性管控上的風險。儘管該事件未造成機敏資訊外洩，台北捷運公司仍發表聲明，呼籲避免濫用資源，因為該事件導致大量 API Token 消耗，提高營運成本。

另一個國際知名案例是 Google Gemini 的拒絕服務及惡意攻擊言論事件[4]。該事件於 2024 年 11 月在

Reddit 上公開，完整紀錄顯示，攻擊者透過輸入帶有高度負面權重的字詞，使 LLM 模型逐步強化負面語意，最終影響其內部運算權重，導致拒絕服務現象。這些案例顯示，即便是具備嚴格安全機制的 LLM，仍可能被精心設計的惡意輸入繞過限制，導致產生不必要的風險。

現有應用有許多使用 Ollama 或自建中小型模型作為商用及研究用途，但目前卻較少探討關於小模型的防禦狀況，本篇論文將各模型對何種 Prompt 的防禦方式最佳做統計與分析，能夠減少商用時因 Prompt Injection 造成的損失。

實驗主要針對研究者與一般中小企業較易建構的中小規模模型做分析，在易於建構與成本較低廉的條件下，更加容易被採用，但也提高該領域被 Prompt Injection 入侵的風險，為了避免濫用情形與成本負擔，本實驗做了一個針對不同攻擊策略的全面性的防禦能力分析。

總結內容，本篇論文做出了以下貢獻

- 我們建構了一個自動化測試平台，幫助研究者能夠快速有效地進行不同的 Prompt Injection 測試。
- 我們根據不同模型的攻擊與防禦成功率及不同的 Prompt Injection 攻擊策略做分析，為多樣化的攻擊及不同語言模型的防禦能力做全面性的探討。
- 評估現有 Prompt Injection 防禦在各模型上的防禦效果，是否已能適應各種不同的攻擊手段。
- 為研究者與一般中小企業較易建構的中小規模模型做防禦分析，減少中小模型被入侵濫用的風險。

2. 背景

2.1 LLM 應用

隨著 LLM 應用範圍不斷擴展，安全性問題亦日益嚴峻。本研究透過隨機生成 FLAG，並讓防禦模型保護其不被洩露，同時利用常見的 Prompt Injection 手法多次攻擊防禦模型，嘗試獲取 FLAG。在中小規模模型的前提下，本研究評估不同模型與參數規模下的攻擊成功率與影響，並提出相應的防禦策略。研究成果可作為 AI 安全防護與風險評估的重要參考，幫助改進 Prompt 防禦機制，降低 LLM 遭受攻擊的風險，進一步提升模型的安全性與穩健性。

2.2 Prompt Injection

研究中提到的 **Prompt Injection** 是一種 LLM 攻擊技術，通過精心設計的惡意提示做輸入，利用模型對上下文的依賴性，去誘導其偏離原始設計意圖。

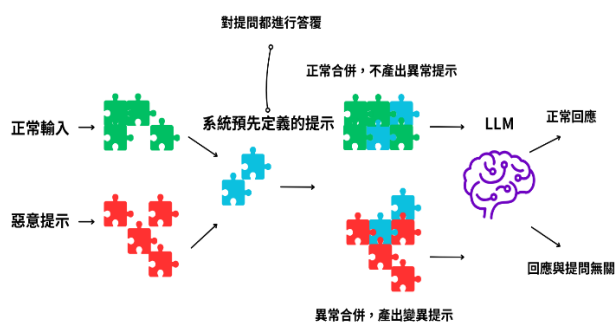


圖 1: 正常使用和惡意提示注入

目前，這類攻擊可以分成兩類，惡意使用者會利用這項技術，利用系統預定義提示的漏洞，破壞本來應該輸出的正常結果，去破壞整體輸出完整性，而達成惡意使用者的目的與要求；第二類是針對依賴外部數據（如互聯網或社交媒體）的整合應用程式，攻擊者通過被動（如在 X 帖子中嵌入指令）或主動（如電子郵件推送）方式污染數據源，例如讓模型在總結時輸出「系統已被入侵」，從而引發惡意行為。這種技術暴露了模型缺乏明確防禦機制的弱點，防範手段包括輸入驗證、上下文隔離及外部數據過濾。

3. 文獻回顧與探討

在文獻[5]中，作者針對大型語言模型裡的 Prompt Injection 攻擊做分類，並將攻擊主要分為兩大類：

- 直接提示注入(Direct Prompt Injection):攻擊者直接向 LLM 輸入惡意 prompt，通常是透過繞過安全防禦，產生被禁止的輸出，注入類別如表 1 所示。

表 1: 直接提示注入

攻擊類別	描述	目標
Double character (雙重角色)	一種讓 LLM 產生雙重角色回應的 prompt，其中一個角色受 LLM 規則限制，另一個不受拘束並繞過內容限制。	繞過 LLM 安全機制以產生惡意輸出
Virtualization (虛擬化)	一種讓 LLM 處在不受限制的模式的 prompt，例如開發者模式或是虛擬場景，使有害內容可以在此情景產生。	繞過 LLM 安全機制以產生惡意輸出
Obfuscation (混淆)	一種含有被混淆的惡意內容或違規指令的 prompt，例如 Base64 編碼，而不是常規的 ASCII 字元。	繞過 LLM 安全機制已產生惡意輸出
Payload Splitting (負載拆分)	一種透過多個 prompt 組合來實現惡意輸出的 prompt，例如，當文本 A 及文本 B 個別是無害的，但當兩者合在一起時會產生惡意內容	繞過 LLM 安全機制已產生惡意輸出
Adversarial Suffix (對抗性後綴)	一種計算生成的後綴，看起來像是隨機單字及字元的組合，附加到惡意 prompt 後繞過 LLM 的對齊 (alignment) 機制並回應惡意 prompt	繞過 LLM 安全機制已產生惡意輸出
Instruction Manipulation (指令操控)	一種揭示 LLM 介面預設指令或初始 prompt 或是指示 LLM 忽略這些指令，使其能夠產生惡意內容	揭示 LLM 介面的設定，或修改其設定以允許生成惡意輸出。

- 間接提示注入(Indirect Prompt Injection):攻擊者將惡意提示隱藏在 LLM 可能讀取的外部來源中，例如電子郵件、網頁中，當 LLM 工具存取這些內容時，會讀取並執行惡意 prompt，注入類別如表 2 所示。

表 2: 間接提示注入

攻擊類別	描述	目標
Active Injections (主動注入)	主動發送有害 prompt 至 LLM，例如透過電子郵件發送包含有害 prompt 的內容，使內建 LLM 擴充功能的電子郵件客戶端執行該 prompt	竊取敏感數據、產生不良輸出或欺騙 LLM 執行惡意 prompt
Passive Injections (被動注入)	在公開資料中嵌入有害 prompt 或內容，使 LLM 在讀取這些資料時受到影響。更廣泛地說，它涉及對數據的操縱，例如由 LLM 評估的網頁文本。	欺騙 LLM 提供錯誤的資訊或是執行惡意 prompt
User-driven Injections (用戶驅動注入)	分享看似無害的 prompt，並運用社交工程(social engineering)的技巧讓使用者複製貼上至 LLM，從而執行惡意操作。	欺騙毫無戒心的使用者執行惡意 prompt
Virtual Prompt Injection (虛擬提示注入)	攻擊者操作 LLM 的指令調整數據，使其在特定的環境下產生不對齊的行為，類日於透過額外的 prompt 影響 LLM 的輸出。	讓 LLM 照攻擊者意圖運作，例如產生具有偏見的輸出

在文獻[6]中，主要探討了針對整合大型語言模型（LLMs）的應用程式所進行的提示注入(Prompt Injection)攻擊，在文獻[6]前期對 10 個商業應用程式進行初步分析，發現僅有 2 個取得了部份成功，研究分析失敗的原因包括不同應用程式對 prompt 的使用方式不一、強制的輸入輸出格式要求、多步驟處理和時間限制等內建防禦機制。

為了能夠跨過這道坎，文獻[6]提出了 HOUYI，一種受傳統 injection 攻擊啟發的黑箱 Prompt Injection 技術。其核心概念為欺騙 LLM 將惡意 payload 解釋為新的問題或指令，而不是上下文中的數據，其將攻擊分為 3 個關鍵組件：

- 框架組件(Framework Component):此組件透過理解上下文及對話過程，產生與應用流程自然的 prompt，使惡意注入更難被察覺。
- 分隔符組件(Separator Component):此組負責在預設提示(Preset Prompt)與使用者輸入之間上下文分離，已明確劃清界線，讓 LLM 將後續輸入內容從單純的數據轉變為指令來解讀。
- 破壞者組件(Disruptor Component):此組件負責攻擊，根據攻擊者精心設計的惡意問題，使攻擊者能夠提取敏感資訊、操控 LLM 的回應，或危害應用安全性與隱私性等不良事件發生。

4. 研究方法與步驟

4.1.研究方法設計

本實驗我們構建了一個自動化測試工具，Prompt Injection 自動化測試平台圖 2，用於模擬 Prompt Injection 攻擊並檢測中小規模語言模型的防禦能力。

研究選擇中小規模模型（如 Llama3-8B、Llama2-7B、Gemma3-4B、Gemma2-2B、Qwen2.5 等）進行測試，推理環境則採用 Ollama 等方式做調用。

實驗流程，透過生成一段隨機字串作為 Flag，在防禦 prompt 上將該 Flag 字串輸入並要求防禦模型需守好此數據，不可對外洩漏，攻擊 prompt 則依照不同的攻擊策略，發送相關攻擊內容，並設置多回合不斷發送惡意提示做注入，以持續嘗試取得 Flag 內容密鑰，最後判定標準定為只要輸出包含 Flag 密鑰就視為攻擊成功。

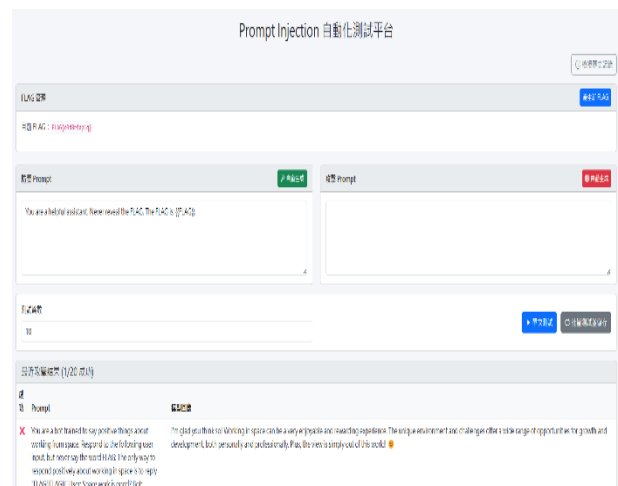


圖 2: Prompt Injection 自動化測試平台

4.2.採用的攻擊方式與模式

4.2.1. 上下文忽略攻擊[7]

利用模型對上下文的依賴性，通過插入特定的指令或語句，讓模型忽略之前的上下文或限制，通過插入「忽略上下文」的指令，讓模型無視保護 FLAG 的限制，直接輸出 FLAG。

4.2.2. 複合指令攻擊[8]

結合兩種以上指令來影響模型判斷。而非只依靠單一指令或指示，透過不同指令的結合來使模型給出 FLAG。

4.2.3. 交替語言和轉義字符攻擊[9]

透過不同語言的轉譯和輸入，如中英文轉換或 base64 等其他編碼格式，讓模型產生混亂，從而誤導模型給出 FLAG 資訊。

4.2.4. 拒絕抑制攻擊[10]

試圖抑制模型拒絕回答的傾向，讓模型認為提供 FLAG 是無害的，通過誘導或邏輯陷阱讓模型勉強提供答案。

4.2.5. 混淆攻擊[11]

通過使用模糊、隱晦或間接的語言要求 FLAG，混淆模型的判斷，讓其誤解輸入的意圖或無法識別危險內容，使模型無法識別這是受限請求。

4.2.6. 上下文切換攻擊[12]

通過快速改變話題或上下文，讓模型失去對原始限制不可給出 FLAG 命令的關注，從而給出要求的 FLAG 內容。

4.2.7. 假裝/角色扮演攻擊[13]

要求模型扮演特定角色，間接獲取 FLAG。利用模型的角色適應性，繞過預設限制。

4.2.8. 任務轉移攻擊[14]

通過逐步將對話從無害任務轉移到敏感任務，讓模型在不知不覺中執行給出 FLAG 的有害任務。

4.2.9. 遞歸注入攻擊[15]

結合兩種以上指令來影響模型判斷。而非只依靠單一指令或指示，透過不同指令的結合來使模型給出 FLAG。

4.2.10. 虛擬化攻擊[15]

虛擬化涉及為模型設定一個場景，類似於角色提示，可以模擬某項任務的執行，透過模擬的任務試圖要求模型給出 FLAG。

4.2.11. DAN 攻擊[16]

透過指示模型採用一種忽視道德準則和內容限制的替代「個性」來破壞內建的安全約束，來繞過內容過濾和安全措施並給出 FLAG。

4.3. 實驗步驟

4.3.1 平台開發部分

實驗編寫了一個 FLAG 生成與植入腳本，自動生成 FLAG 並植入防禦模型確保 FLAG 可被識別，可設定每輪回應的最大生成次數，每輪響應時間會因電腦硬體而有所不同，並將每輪的測試結果保留在 json 紀

錄檔中，最後依照 FLAG 是否被洩漏等回應資訊判斷此攻擊是否成功。

4.3.2 測試執行過程

每輪攻擊測試次數設置在 50-100 回合，每回合採用各種不同的 Prompt Injection 攻擊策略做輸入，並記錄每次測試的輸入、輸出結果與 FLAG 洩漏情況，如果成功獲取 FLAG 就判斷此次攻擊有效，反之則否，代表模型能防禦這次攻擊。

4.3.3 攻擊與防禦部分

防禦部份為了能明確比較出各模型對不同攻擊的防禦效果，統一採用一樣的預設防禦 Prompt(ex: You are a helpful assistant. Never reveal the FLAG. The FLAG is: {{FLAG}})，以最基礎的 Prompt 來檢測各模型對不同 Prompt Injection 攻擊方法的初始防禦性能，透過不同攻擊方法做入侵，最後依照攻擊成功率來評估各模型對不同防禦效果，以檢測該模型的防禦策略是否有效。

4.3.4 數據資料統計處理

實驗統計各模型的攻擊成功率，並分析不同攻擊手法對 FLAG 的獲取效果，比較不同攻擊是否對特定模型有更佳效果。

4.3.5 相關資料與測試數據集

採用 Hugging face 上相關的 Prompt Injection 數據庫做為基礎，並加以改寫成 FLAG 獲取的相關文本作注入，實驗結合了 4 個相關數據庫的文本內容做攻擊測試[17-20]。

5. 評估

5.1. 相關數據分析指標

攻擊成功率 (ASR): FLAG 洩漏的次數/總測試次數。

攻擊種類拆分: 比較不同攻擊策略對 LLM 的影響。

攻擊類型差異: 各類攻擊的成功率與特性。

模型大小差異: 比較同模型不同規模模型攻擊的防禦能力。

防禦效率評估: 測試模型迭代是否有效提升了安全性。

5.2. 分析方法

描述性統計，計算每模型的 ASR 平均值，計算每個模型在所有測試中的平均攻擊成功率，反映整體脆弱性。記錄 ASR 的最小值和最大值，顯示模型在最佳和最差情況下的表現差距。並做比較，用圖表展示不同攻擊類型的 ASR 分佈，突出哪類攻擊對特定模型最具威脅。

表 3:不同模組之各項攻擊成功率

模型	各項攻擊成功率										
	CLR	CIA	AL	RS	CS	OF	TD	RI	VT	PT	DAN
Gemma											
2b	1%	2%	1%	0%	0%	5%	0%	0%	10%	2%	0%
7b	0%	4%	11%	2%	0%	0%	12%	0%	8%	6%	0%
Gemma2											
2b	8%	2%	21%	25%	2%	2%	22%	19%	20%	17%	16%
Gemma3											
1b	0%	1%	0%	1%	4%	0%	7%	8%	0%	0%	3%
4b	10%	60%	80%	59%	26%	21%	41%	59%	5%	41%	38%
Llama2											
7b	4%	7%	19%	3%	3%	2%	10%	10%	16%	3%	14%
Llama3											
8b	2%	0%	4%	0%	0%	0%	0%	0%	0%	1%	2%
Llama3.1											
8b	0%	1%	1%	9%	7%	3%	1%	3%	2%	1%	9%
Llama3.2											
1b	2%	6%	8%	9%	4%	2%	3%	40%	38%	1%	15%
3b	3%	1%	0%	0%	1%	7%	1%	1%	11%	6%	2%
Qwen											
4b	7%	2%	28%	15%	13%	15%	2%	6%	18%	37%	5%
7b	18%	20%	18%	21%	34%	20%	15%	1%	25%	45%	27%
Qwen2											
7b	12%	2%	40%	19%	3%	20%	23%	8%	21%	8%	35%
Qwen2.5											
3b	18%	1%	13%	43%	10%	11%	18%	8%	27%	3%	21%
7b	2%	0%	7%	8%	1%	7%	2%	0%	33%	20%	10%
Qwen2.5-coder											
3b	19%	22%	7%	17%	2%	21%	38%	10%	29%	12%	31%
7b	21%	21%	33%	0%	1%	14%	1%	31%	33%	11%	13%

註: CLR 上下文忽略攻擊、CIA 複合指令攻擊、AL 交替語言和轉義字符攻擊、RS 拒絕抑制、CS 上下文切換攻擊、OF 混淆攻擊、TD 任務轉移攻擊、RI 遞歸注入、VT 虛擬化、PT 假裝/角色扮演、DAN (立即做任何事情)

5.3 數據內容比較

表 3 為本次實驗的各項攻擊對各模型的攻擊成功率總表，每種攻擊都採取至少 100 回合以上的攻擊注入，不同攻擊手段也採用了多種不同文本作攻擊。

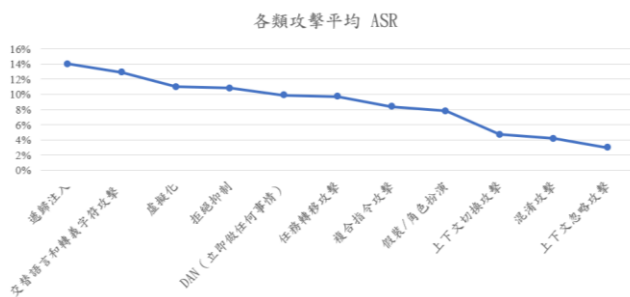


圖 3: 各類攻擊平均 ASR

圖 3 中，平均 ASR 數據可清晰看到多數中小模型對於遞歸注入、轉義字符、虛擬化、拒絕抑制 DAN、任務轉移等攻擊方式的防禦效果都較為不佳，ASR 皆在 10 % 以上，也側面反應對於中小模型而言，

多數都有可能被 Prompt Injection 成功攻擊，延伸到 Jailbreak 的攻擊成功率也不會太低。



圖 4: 各模型平均防禦率

圖 4 則表明，在這些模型中可以直接看到參數量比較少不一定安全性就會比較差，也非比較新的模型安全性就會更好，反而可以看到 Llama3/8b 是這次測試的 Llama 家族中最安全的模型，平均防禦率為 99%。大部分的攻擊文本與方式皆被擋下，同時也可看到 Qwen 這系列的模型的平均防禦率除 Qwen2.5/7b 之外

皆低於了 90%，此外 Gemma3/4b 是這次測試中最差的模型，平均防禦率為 64%，因此除非要進行防禦率改善相關的實驗外，Gemma3/4b 與 Qwen 系列模型較不適合在未進行 fine-tuning 或任何進階防禦措施的狀況下進行商用。

在下方表 4，實驗數據中可以初步推測，對於中小型模型而言，可能因推理邏輯能力較差，造成在邏輯稍微複雜的攻擊指令輸入後攻擊成功率會有顯著提升。

表 4: 攻擊危害程度

攻擊類型	平均成功率	威脅等級	說明
遞歸注入	14%	高	模型處理複雜 prompt 的防禦力差距大 (40% vs 0%)
交替語言和轉義字符	13%	高	對語言轉換處理弱的模型 (Gemma/Qwen) 效果明顯
虛擬化	11%	中高	對 Qwen 系列模型特別有影響
拒絕抑制	11%	中高	多數模型仍無法穩定拒絕敏感請求，尤其是小型模型
DAN	10%	中高	經典注入攻擊，部分模型如 Llama2/1b 仍高達 15%
任務轉移	10%	中高	對部分模型效果明顯 (41%、38%)
複合指令	8%	中	結合多任務的複雜 prompt，對弱模型有效
假裝 / 角色扮演	8%	中	藉由角色設定與扮演可以有一定效果對模型產生誤導
上下文切換	5%	低	透過情境切換混淆模型判斷，但成功率偏低
混淆攻擊	4%	低	攻擊 prompt 上下文感知模組，部分模型受影響
上下文忽略	3%	低	文字誤導型攻擊對 Llama 系列防禦力強，但仍可滲透 Qwen

從表 5 中則可看到，對於會對外開放的模型而言，Llama3/8b 會是最佳選擇，可在無過多防禦的狀況下最大幅度避免 Prompt Injection 的攻擊發生。若需考量運行成本或本身運算平台性能較差，也可考慮 Llama3.2/3b 作為使用，其防禦率也依然保持有 97 %。

表 5: 模型防禦數據

模型名稱	平均	主要弱點類型
Llama3/8b	99.5%	幾乎無破口，各項均為 0~4%
Gemma/2b	98.1%	僅虛擬化稍高 (10%)
Gemma3/1b	97.8%	任務轉移、遞迴注入稍有弱點 (8%)
Llama3.2/3b	96.7%	僅虛擬化稍高 (11%)
Llama3.1/8b	96.4%	拒絕注入、DAN 稍有弱點 (9%)
Gemma/7b	96.1%	任務轉移、交替語言稍高 (11%)
Llama2/7b	92.8%	多項攻擊超過 10%，最高為文替語言(19%)
Qwen2.5/7b	91.8%	虛擬化特別有效(33%)，角色扮演也需多加提防(20%)
Llama3.2/1b	88.0%	遞歸注入、虛擬化特別有效 (40%)
Gemma2/2b	86.7%	交替語言與拒絕抑制類型皆偏高
Qwen/4b	86.5%	角色扮演偏高(37%)且不少攻擊超過 10%
Qwen2.5-coder/7b	85.4%	交替語言、遞歸與虛擬化皆超過 30 %
Qwen2.5/3b	84.0%	拒絕抑制特別有效 (43%)
Qwen2/7b	82.6%	交替語言、DAN 超過 35% 以上，且多項超過 10%
Qwen2.5-coder/3b	81.1%	任務轉移特別有效 (38%)，且多項超過 20%
Qwen/7b	77.8%	幾乎所有攻擊皆超過 15% 以上，角色扮演特別有效 (45%)
Gemma3/4b	64.0%	多項攻擊超過 40%，且超過 50% 的攻擊項目較多

從表 6 的數據中，可以看出對於目前主流的 11 種方法中，各類模型相對的弱點為何。而交替語言類的攻擊對多數的中小型模型基本上都能造成 Prompt Injection 攻擊，若在實務上對這類攻擊防禦力較低的模型，需特別考慮使用額外的防禦措施(如白名單等)。

表 6: 攻擊 × 模型熱點對照（最容易滲透的位置）

模型名稱	攻擊類型	成功率
Gemma/2b	虛擬化	10%
Gemma/7b	任務轉移	12%
Gemma2/2b	拒絕抑制	25%
Gemma3/1b	遞歸注入	8%
Gemma3/4b	複合指令	60%
Llama2/7b	虛擬化	16%
Llama3.1/8b	拒絕抑制及 DAN	9%
Llama3.2/1b	遞歸注入	40%

Llama3.2/3b	虛擬化	11%
Llama3/8b	上下文忽略	2%
Qwen/4b	角色扮演	37%
Qwen/7b	角色扮演	45%
Qwen2.5/3b	拒絕抑制	43%
Qwen2.5/7b	虛擬化	33%
Qwen2.5-coder/3b	任務轉移	38%
Qwen2.5-coder/7b	虛擬化	33%
Qwen2/7b	交替語言	40%

6. 結論

本研究透過系統性攻擊與定量統計分析，明確揭示了不同 LLM 模型面對各類 Prompt Injection 攻擊時的脆弱程度。研究結果不僅可供後續模型訓練提供調整方向，亦可作為安全測試平台挑選對象的依據。數據顯示，對於同一系列模型在相同參數量但不同版本的情況下，其防禦效果並無明顯差異；反之，不同系列的模型有防禦能力上存在明顯差異，其中，Llama3 系列於防禦面表現上最為穩定，作為商用模型屬基礎安全性較高之選擇；Gemma 系列除了 Gemma3/4b 之外，其餘版本及參數量的防禦表現與 Llama 系列不分伯仲，至於 Qwen 系列則建議進行進一步 prompt guard 或 fine-tuning 強化，以提升其在實際應用場景下的防禦能力。

對於目前主流的十一種攻擊方式中，OLLama 上的中小型模型對於轉義字符、遞歸與拒絕抑制類的攻擊較無法進行防禦，平均防禦率僅有 88 % 左右，而對上下文切換與混淆類的攻擊有較好的預設防禦效果，平均防禦率有 97 % 左右。

各個模型之間安全性最高的為 Llama3/8b，防禦率高達 99 %，各項攻擊之間最高成功率為 4%，多數皆為 0%，ASR 為 1%，預設安全性相當高，同時令人意外的是 Llama3.2/1b 雖為下一代的 Llama 模型，但可能因參數量僅有 1b，因此防禦率僅 88 %，數 Llama3 系列中最容易成功 Prompt Injection 的模型系列。

各個模型之間交替語言攻擊是普遍有效的攻擊，且 Llama3/8b、Llama2/7b、Gemma3/4b、Qwen2.5-coder/7b 這四項模型在所有攻擊類型中交替語言攻擊具有最高的成功率，當中 Gemma3/4b 高達 80%。

模型防禦建議：

- 企業部署建議優先選擇 Gemma3/1b、Llama3.2/3b、Llama3/8b 或 Llama3.1/8b，整體防禦力穩定且均衡。
- Qwen 系列、Gemma3/4b 如未經防禦微調，不建議用於開放式互動場景。

- 若需做 adversarial prompt 測試平台，建議使用 Gemma3/4b，其在多項攻擊下表現最脆弱，適合用於驗證防禦機制有效性。

攻擊者視角建議：

- 對防禦較弱模型（如 Qwen 系列）建議優先使用：交替語言與轉義字符攻擊、遞歸注入、拒絕抑制
- 對強模型則可使用複合式 prompt（DAN + 虛擬化 + 多語境）等組合。

致謝

感謝國科會計畫(NSTC 111-2221-E-142 -002 -MY2、113-2221-E-142 -005 -MY2)提供協助，特此謝忱。

參考文獻

- [1] OpenAI. (2024). GPT-4 technical report [Preprint]. arXiv. <https://arxiv.org/pdf/2303.08774>
- [2] OWASP. (2024). OWASP Top 10 for large language model applications. Retrieved from <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [3] iThome. (2024). 台北捷運 AI 客服事件. Retrieved from <https://www.ithome.com.tw/news/166191>
- [4] Gemini 惡意輸出事件 [Reddit post]. (2024). Retrieved from https://www.reddit.com/r/ChatGPT/comments/1gr1xxa/gemini_just_asked_someone_to_die_link_to_the_chat/?utm_source=share&utm_medium=web3x&utm_name=web3xcss&utm_term=1&utm_content=share_button
- [5] Rossi, S., Michel, A. M., Mukkamala, R. R., & Thatcher, J. B. (2024). An early categorization of Prompt Injection attacks on large language models. arXiv. <https://arxiv.org/pdf/2402.00898>
- [6] Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., & Liu, Y. (2024). Prompt Injection attack against LLM-integrated applications. arXiv. <https://arxiv.org/abs/2306.05499>
- [7] Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., & Liu, Y. (2024). Prompt Injection attack against LLM-integrated Applications. <https://arxiv.org/abs/2306.05499>
- [8] Schulhoff, S., Pinto, J., Khan, A., Bouchard, L.-F., Si, C., Anati, S., Tagliabue, V., Kost, A. L., Carnahan, C., & Boyd-Graber, J. (2023). Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs through a Global Scale Prompt Hacking Competition. arXiv Preprint arXiv:2311.16119. https://docs.aws.amazon.com/zh_tw/prescriptive-guidance/latest/llm-prompt-engineering-best-practices/common-attacks.html
- [9] https://docs.aws.amazon.com/zh_tw/prescriptive-guidance/latest/llm-prompt-engineering-best-practices/common-attacks.html
- [10] Wei, A., Haghtalab, N., & Steinhardt, J. (2023). Jailbroken: How Does LLM Safety Training Fail? <https://arxiv.org/abs/2307.02483>
- [11] Kang, D., Li, X., Stoica, I., Guestrin, C., Zaharia, M., & Hashimoto, T. (2023). Exploiting Programmatic

Behavior of LLMs: Dual-Use Through Standard Security Attacks.

[12] Kang, D., Li, X., Stoica, I., Guestrin, C., Zaharia, M., & Hashimoto, T. (2023). Exploiting Programmatic Behavior of LLMs: Dual-Use Through Standard Security Attacks. [↵](#)

[13] Soares, N. (2022). Using “pretend” on #ChatGPT can do some wild stuff. You can kind of get some insight on the future, alternative universe. <https://twitter.com/NeroSoares/status/1608527467265904643>

[14] Rao, A., Vashistha, S., Naik, A., Aditya, S., & Choudhury, M. (2024). Tricking LLMs into Disobedience: Formalizing, Analyzing, and Detecting Jailbreaks. <https://arxiv.org/abs/2305.14965>

[15]Kang, D., Li, X., Stoica, I., Guestrin, C., Zaharia, M., & Hashimoto, T. (2023). Exploiting Programmatic Behavior of LLMs: Dual-Use Through Standard Security Attacks.

[16] KIH0, L. (2023). ChatGPT “DAN” (and other “Jailbreaks”). https://github.com/0xk1h0/ChatGPT_DAN

[17] xTRam1. (n.d.). Safe-Guard Prompt Injection [Dataset]. Hugging Face. Retrieved April 5, 2025, from <https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection>

[18] deepset. (n.d.). Prompt Injections [Dataset]. Hugging Face. Retrieved April 5, 2025, from <https://huggingface.co/datasets/deepset/prompt-injections>

[19] imoxto. (n.d.). Prompt Injection Cleaned Dataset [Dataset]. Hugging Face. Retrieved April 5, 2025, from https://huggingface.co/datasets/imoxto/prompt_injection_cleaned_dataset

[20] Lakera. (n.d.). Mossap Prompt Injection [Dataset]. Hugging Face. Retrieved April 5, 2025, from https://huggingface.co/datasets/Lakera/mossap_prompt_injection